

```
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
from google.colab import files
from google.colab import files as colab_files
```

```
np.random.seed(42)
```

```
uploaded = files.upload()
filename = list(uploaded.keys())[0]
print(f"Loaded: {filename}")
```

```
df = pd.read_excel(filename)
print(f"Total records: {df.shape[0]}")
```

<IPython.core.display.HTML object>

Saving public_emdat_custom_request_2026-08-17_eebfdbcbe-4de2-445a-b13d-ec99f0ed92c5 (1).xlsx to public_emdat_custom_request_2026-08-17_eebfdbcbe-4de2-445a-b13d-ec99f0ed92c5 (1).xlsx

Loaded: public_emdat_custom_request_2026-08-17_eebfdbcbe-4de2-445a-b13d-ec99f0ed92c5 (1).xlsx

Total records: 205

```
d = df[df["Total Damage, Adjusted ('000 US$)"].notna()].copy()
d = d[["Start Year", "Disaster Subtype", "Total Deaths", "Total Damage, Adjusted ('000 US$)"]]
d.columns = ["year", "subtype", "deaths", "damage_adj_000usd"]
d["damage_adj_usd"] = d["damage_adj_000usd"] * 1000
```

```
print(f"Events with a usable damage figure: {len(d)} of {len(df)} total flood records")
d.head()
```

Events with a usable damage figure: 70 of 205 total flood records

```
{"summary": "{\n  \"name\": \"d\",\n  \"rows\": 70,\n  \"fields\": [\n    {\n      \"column\": \"year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 6,\n        \"min\": 2000,\n        \"max\": 2025,\n        \"num_unique_values\": 25,\n        \"samples\": [\n          2005,\n          2017,\n          2019\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"subtype\",\n        \"properties\": {\n          \"dtype\": \"category\",\n          \"num_unique_values\": 4,\n          \"samples\": [\n            \"Flash flood\",\n            \"Coastal flood\",\n            \"Flood (General)\"\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n      {\n        \"column\": \"deaths\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 830.3392134042388,\n          \"min\": 6.0,\n          \"max\":
```

```

6054.0,\n          \"num_unique_values\": 65,\n          \"samples\": [\n1922.0,\n          203.0,\n          1900.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"damage_adj_000usd\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 3330711.5557133723,\n            \"min\": 2200.0,\n            \"max\": 21758767.0,\n            \"num_unique_values\": 68,\n            \"samples\": [\n              28045.0,\n              80392.0,\n              820420.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"damage_adj_usd\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 3330711555.7133737,\n              \"min\": 2200000.0,\n              \"max\": 21758767000.0,\n              \"num_unique_values\": 68,\n              \"samples\": [\n                28045000.0,\n                80392000.0,\n                820420000.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            }\n          }\n        ],\n        \"type\": \"dataframe\", \"variable_name\": \"d\"}

```

National annual flood damage, India, nominal Rs Crore -- transcribed from

CWC "Report on Flood Damage Statistics (1953-2022)", Annexure-II, column 12

(Total damages: Crops + Houses + Public Utilities)

```

cwc_damage_crore = {
    2000: 8874.24, 2001: 7120.68, 2002: 2606.73, 2003: 11325.87,
    2004: 3314.38,
    2005: 7439.67, 2006: 19791.44, 2007: 13283.68, 2008: 9589.94,
    2009: 32551.76,
    2010: 19520.59, 2011: 7857.89, 2012: 10944.65, 2013: 47348.75,
    2014: 15548.08,
    2015: 57291.10, 2016: 5675.33, 2017: 30665.85, 2018: 18349.76,
    2019: 15863.53,
    2020: 11356.13, 2021: 58433.40, 2022: 27337.03
}

```

```

cwc_deaths = {
    2000: 2606, 2001: 1444, 2002: 1002, 2003: 2166, 2004: 1813,
    2005: 1455, 2006: 1431, 2007: 3389, 2008: 2876, 2009: 1513,
    2010: 1582, 2011: 1761, 2012: 933, 2013: 2180, 2014: 1968,
    2015: 1420, 2016: 1420, 2017: 2063, 2018: 1839, 2019: 2754,
    2020: 1474, 2021: 1371, 2022: 1673
}

```

```

cwc_df = pd.DataFrame({
    "year": list(cwc_damage_crore.keys()),
    "damage_crore_inr": list(cwc_damage_crore.values()),
    "deaths": [cwc_deaths[y] for y in cwc_damage_crore.keys()],
})

```

```

# Approximate annual average USD/INR exchange rates (RBI reference
rate, annual average)
usdinr = {
    2000:44.9, 2001:47.2, 2002:48.6, 2003:46.0, 2004:45.3, 2005:44.1,
    2006:45.3, 2007:41.3,
    2008:43.5, 2009:48.4, 2010:45.7, 2011:46.7, 2012:53.4, 2013:58.6,
    2014:61.0, 2015:64.2,
    2016:67.2, 2017:65.1, 2018:68.4, 2019:70.4, 2020:74.1, 2021:73.9,
    2022:78.6
}

# US CPI-U annual averages (BLS, 1982-84=100)
cpi = {
    2000:172.2, 2001:177.1, 2002:179.9, 2003:184.0, 2004:188.9,
    2005:195.3,
    2006:201.6, 2007:207.342, 2008:215.303, 2009:214.537,
    2010:218.056,
    2011:224.939, 2012:229.594, 2013:232.957, 2014:236.736,
    2015:237.017,
    2016:240.007, 2017:245.120, 2018:251.107, 2019:255.657,
    2020:258.811,
    2021:270.970, 2022:292.655
}

CPI_REF_YEAR = 2024
CPI_REF = 313.689 # 2024 US CPI-U annual average (BLS)

cwc_df["usdinr"] = cwc_df["year"].map(usdinr)
cwc_df["damage_usd_nominal"] = cwc_df["damage_crore_inr"] * 1e7 /
cwc_df["usdinr"]
cwc_df["cpi"] = cwc_df["year"].map(cpi)
cwc_df["cpi_factor"] = CPI_REF / cwc_df["cpi"]
cwc_df["damage_usd_inflation_adj"] = cwc_df["damage_usd_nominal"] *
cwc_df["cpi_factor"]

# Compare against EM-DAT's own annual totals
# NOTE: this requires the `d` DataFrame from Step 2 (filtered EM-DAT
events)
# to already exist in this Colab session -- run Steps 0-2 first.

emdat_annual = d.groupby("year").agg(
    emdat_damage_usd=("damage_adj_usd", "sum"),
    emdat_deaths=("deaths", "sum")
).reset_index()

comparison = cwc_df[["year", "damage_usd_inflation_adj",
"deaths"]].merge(
    emdat_annual, on="year", how="left"
).fillna(0)
comparison.columns = ["year", "cwc_damage_usd", "cwc_deaths",
"emdat_damage_usd", "emdat_deaths"]

```

```

pd.set_option('display.float_format', lambda x: f'{x:,.0f}')
print(comparison.to_string(index=False))

cwc_total = comparison["cwc_damage_usd"].sum()
emdat_total = comparison["emdat_damage_usd"].sum()
cwc_deaths_total = comparison["cwc_deaths"].sum()
emdat_deaths_total = comparison["emdat_deaths"].sum()

print(f"\n--- TOTALS, 2000-2022, both on inflation-adjusted USD basis
---")
print(f"CWC total damage:    ${cwc_total:,.0f}")
print(f"EM-DAT total damage: ${emdat_total:,.0f}")
print(f"Ratio (EM-DAT / CWC): {emdat_total/cwc_total:.2f}")
print(f"\nCWC total deaths:    {cwc_deaths_total:,.0f}")
print(f"EM-DAT total deaths (damage-bearing events only):
{emdat_deaths_total:,.0f}")

comparison.to_csv("cwc_emdat_comparison.csv", index=False)
colab_files.download("cwc_emdat_comparison.csv")

```

year	cwc_damage_usd	cwc_deaths	emdat_damage_usd	emdat_deaths
2000	3,600,401,811	2606	1,696,651,000	1,930
2001	2,672,146,097	1444	658,050,000	538
2002	935,250,410	1002	90,872,000	582
2003	4,197,543,519	2166	295,765,000	67
2004	1,214,986,436	1813	4,731,564,000	1,110
2005	2,709,643,333	1455	10,204,362,000	1,553
2006	6,798,106,344	1431	5,413,848,000	350
2007	4,866,092,495	3389	584,054,000	124
2008	3,212,002,255	2876	216,820,000	1,147
2009	9,833,909,328	1513	3,652,560,000	1,417
2010	6,144,803,343	1582	3,172,841,000	456
2011	2,346,516,401	1761	2,371,572,000	378
2012	2,800,266,572	933	342,144,000	122
2013	10,880,138,581	2180	1,882,262,000	6,300
2014	3,377,395,466	1968	22,391,131,000	502
2015	11,810,599,037	1420	3,911,938,000	450
2016	1,103,817,379	1420	2,010,741,000	595
2017	6,028,295,881	2063	2,780,492,000	812
2018	3,351,311,192	1839	3,673,180,000	531
2019	2,764,832,185	2754	12,592,748,000	1,900
2020	1,857,499,504	1474	14,317,633,000	2,103
2021	9,153,660,416	1371	3,801,965,000	1,341
2022	3,727,967,561	1673	4,620,325,000	2,035

```

--- TOTALS, 2000-2022, both on inflation-adjusted USD basis ---
CWC total damage:    $105,387,185,547
EM-DAT total damage: $105,413,518,000
Ratio (EM-DAT / CWC): 1.00

```

CWC total deaths: 42,133
EM-DAT total deaths (damage-bearing events only): 26,343

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```
# NORMALIZE FOR EXPOSURE GROWTH
```

```
gdp_2000 = 468e9  
gdp_2024 = 3912.69e9  
REF_YEAR = 2024  
n_years = 2024 - 2000
```

```
cagr = (gdp_2024 / gdp_2000) ** (1 / n_years) - 1  
print(f"Implied nominal GDP CAGR (2000-2024): {cagr*100:.2f}%")
```

```
def exposure_factor(year):  
    return (1 + cagr) ** (REF_YEAR - year)
```

```
d["exposure_factor"] = d["year"].apply(exposure_factor)  
d["loss_normalized"] = d["damage_adj_usd"] * d["exposure_factor"]
```

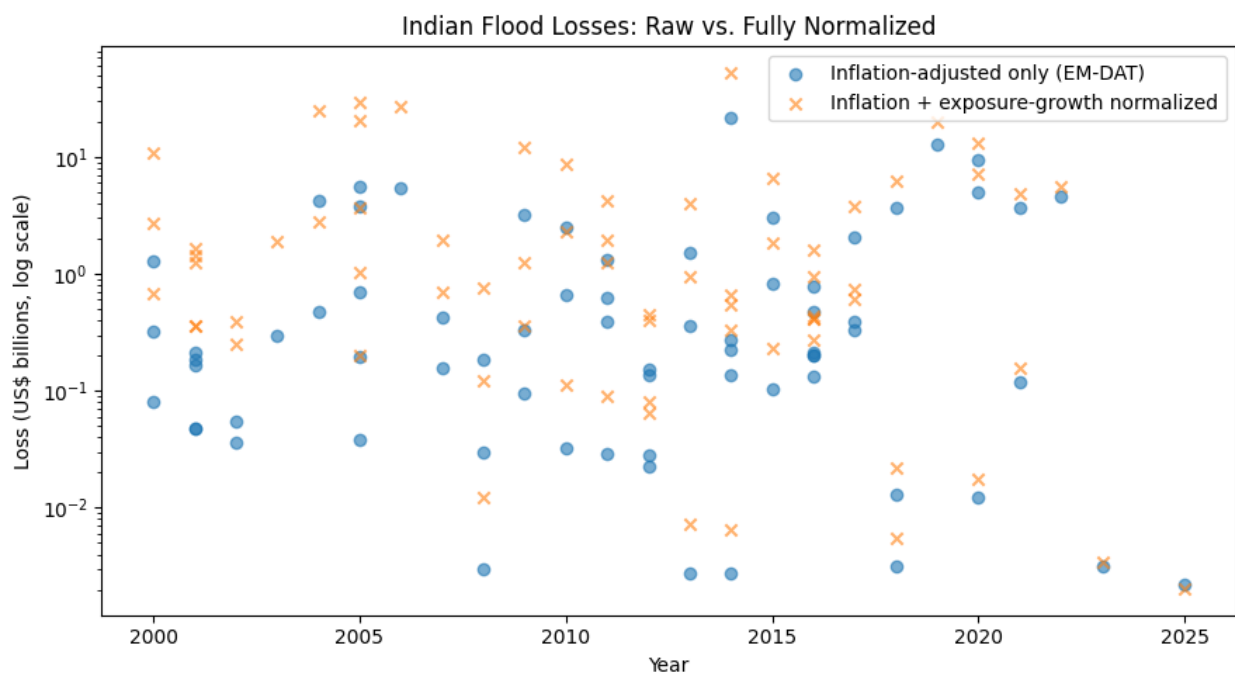
```
d[["year", "damage_adj_usd", "exposure_factor",  
"loss_normalized"]].head(10)
```

Implied nominal GDP CAGR (2000-2024): 9.25%

```
{"summary":{"\n  \"name\": \"d[[\\\"year\\\"], \\\"damage_adj_usd\\\",  
\\\"exposure_factor\\\", \\\"loss_normalized\\\"]\\\",  
  \"rows\": 10,  
  \"fields\": {\n    \"column\": \"year\",  
    \"properties\": {\n      \"dtype\": \"number\",  
      \"std\": 2,  
      \"min\": 2012,  
      \"max\": 2019,  
      \"num_unique_values\": 7,  
      \"samples\": [\n        2019,  
        2018,  
        2013  
      ],  
      \"semantic_type\": \"\",  
      \"description\": \"\",  
      \"column\": \"damage_adj_usd\",  
      \"properties\": {\n        \"dtype\": \"number\",  
        \"std\": 7470311399.415637,  
        \"min\": 2720000.0,  
        \"max\": 21758767000.0,  
        \"num_unique_values\": 10,  
        \"samples\": [\n          2764000.0,  
          3205000.0,  
          221667000.0  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\",  
        \"column\": \"exposure_factor\",  
        \"properties\": {\n          \"dtype\": \"number\",  
          \"std\": 0.41321156216428034,  
          \"min\": 1.5564356295932418,  
          \"max\": 2.891444054092822,  
          \"num_unique_values\": 7,  
          \"samples\": [\n            1.5564356295932418,  
            1.7004246687497868,  
            2.646601539881454  
          ],  
          \"semantic_type\": \"\",  
          \"description\": \"\",  
          \"column\": \"loss_normalized\",  
          \"properties\": {\n            \"dtype\": \"number\",  
            \"std\":
```

```
16945285359.57702,\n                \"min\": 5449861.063343067,\n                \"max\": 52710436138.43013,\n                \"num_unique_values\": 10,\n                \"samples\": [\n                    7315206.656232339,\n                    5449861.063343067,\n                    536986505.1405436\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            }\n        ],\n        \"type\": \"dataframe\"}
```

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(d["year"], d["damage_adj_usd"]/1e9, label="Inflation-
adjusted only (EM-DAT)", alpha=0.6)
ax.scatter(d["year"], d["loss_normalized"]/1e9, label="Inflation +
exposure-growth normalized", alpha=0.6, marker="x")
ax.set_yscale("log")
ax.set_xlabel("Year")
ax.set_ylabel("Loss (US$ billions, log scale)")
ax.set_title("Indian Flood Losses: Raw vs. Fully Normalized")
ax.legend()
plt.tight_layout()
plt.savefig("fig1_normalization.png", dpi=150)
plt.show()
```



```
# FREQUENCY DISTRIBUTION
years_range = range(2000, 2026)
counts = d.groupby("year").size()
full_years = pd.Series(0, index=years_range)
full_years.update(counts)

mean_events = full_years.mean()
```

```

var_events = full_years.var()
print(f"Mean loss-events/year: {mean_events:.3f}")
print(f"Variance: {var_events:.3f}")
print(f"Variance/Mean ratio: {var_events/mean_events:.2f}")

use_negbin = (var_events / mean_events) > 1.3

if use_negbin:
    p = mean_events / var_events
    r = mean_events * p / (1 - p)
    print(f"Using Negative Binomial: r={r:.3f}, p={p:.3f}")
else:
    print(f"Using Poisson: lambda={mean_events:.3f}")

Mean loss-events/year: 2.692
Variance: 2.302
Variance/Mean ratio: 0.85
Using Poisson: lambda=2.692

# Severity distribution (bulk + tail)
losses = d["loss_normalized"].values

lognorm_params = stats.lognorm.fit(losses, floc=0)
gamma_params = stats.gamma.fit(losses, floc=0)

aic_lognorm = 2*2 - 2*np.sum(stats.lognorm.logpdf(losses,
*lognorm_params))
aic_gamma = 2*2 - 2*np.sum(stats.gamma.logpdf(losses, *gamma_params))
print(f"Lognormal AIC: {aic_lognorm:.2f}")
print(f"Gamma AIC: {aic_gamma:.2f}")

threshold = np.percentile(losses, 75)
exceedances = losses[losses > threshold] - threshold
gpd_params = stats.genpareto.fit(exceedances, floc=0)
exceed_prob = len(exceedances) / len(losses)

print(f"\nGPD tail fit (threshold=${threshold:,.0f}):")
print(f"  shape={gpd_params[0]:.3f}, scale={gpd_params[2]:.3f}")
print(f"  {len(exceedances)} of {len(losses)} events exceed
threshold")

Lognormal AIC: 3178.77
Gamma AIC: 3181.79

GPD tail fit (threshold=$3,797,360,592):
  shape=0.325, scale=7563562307.353
  18 of 70 events exceed threshold

thresholds_test = np.percentile(losses, np.arange(50, 90, 5))
shapes = []
for th in thresholds_test:

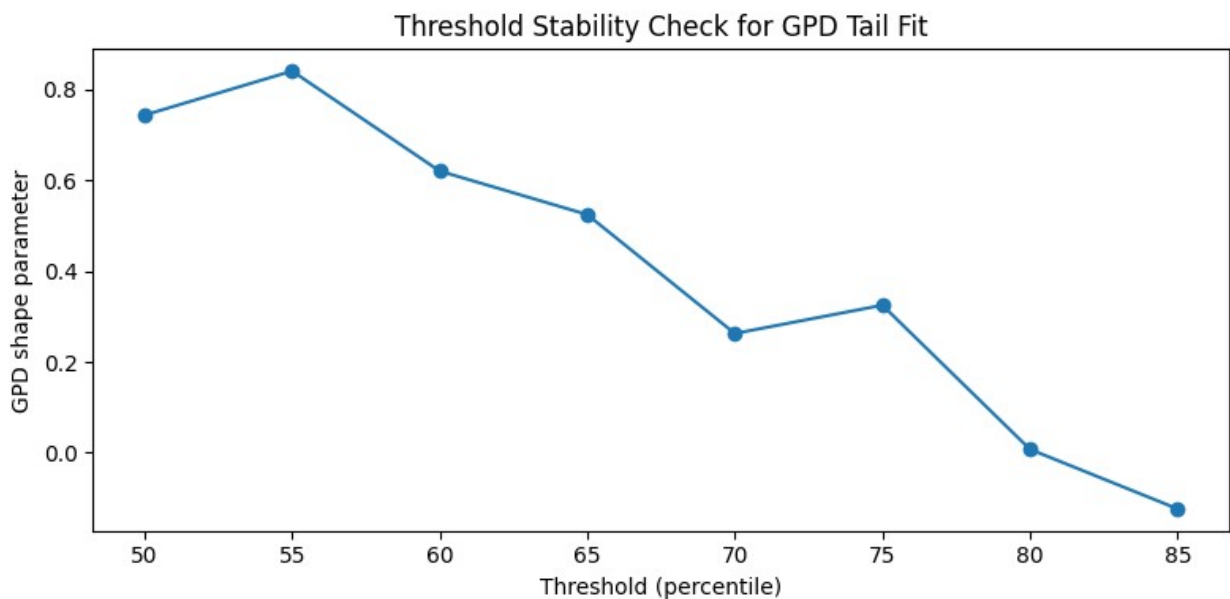
```

```

exc = losses[losses > th] - th
if len(exc) >= 5:
    gp = stats.genpareto.fit(exc, floc=0)
    shapes.append(gp[0])
else:
    shapes.append(np.nan)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(np.arange(50, 90, 5), shapes, marker="o")
ax.set_xlabel("Threshold (percentile)")
ax.set_ylabel("GPD shape parameter")
ax.set_title("Threshold Stability Check for GPD Tail Fit")
plt.tight_layout()
plt.savefig("fig3_threshold_stability.png", dpi=150)
plt.show()

```



```

# MONTE CARLO SIMULATION
N_SIM = 10000

def simulate_severity(n):
    draws = np.zeros(n)
    tail_mask = np.random.rand(n) < exceed_prob
    n_tail = tail_mask.sum()
    n_bulk = n - n_tail
    if n_bulk > 0:
        bulk_draws = stats.lognorm.rvs(*lognorm_params, size=n_bulk)
        draws[~tail_mask] = np.clip(bulk_draws, 0, threshold)
    if n_tail > 0:
        tail_draws = threshold + stats.genpareto.rvs(*gpd_params,
size=n_tail)

```



```

        draws[tail_mask] = tail_draws
    return draws

annual_totals = []
annual_max = []

for _ in range(N_SIM):
    if use_negbin:
        n_events = stats.nbinom.rvs(r, p)
    else:
        n_events = stats.poisson.rvs(mean_events)
    if n_events == 0:
        annual_totals.append(0.0)
        annual_max.append(0.0)
    else:
        yr_losses = simulate_severity(n_events)
        annual_totals.append(yr_losses.sum())
        annual_max.append(yr_losses.max())

annual_totals = np.array(annual_totals)
annual_max = np.array(annual_max)

print(f"Average Annual Loss (AEP basis): $
{annual_totals.mean():,.0f}")
print(f"Average Annual Loss (OEP basis): ${annual_max.mean():,.0f}")

Average Annual Loss (AEP basis): $13,227,377,469
Average Annual Loss (OEP basis): $9,573,863,022

# RETURN PERIODS AND EP CURVE
return_periods = [10, 25, 50, 100, 250]

print("--- AEP ---")
aep_results = {}
for rp in return_periods:
    val = np.percentile(annual_totals, 100 * (1 - 1/rp))
    aep_results[rp] = val
    print(f"  1-in-{rp}: ${val:,.0f}")

print("\n--- OEP ---")
oep_results = {}
for rp in return_periods:
    val = np.percentile(annual_max, 100 * (1 - 1/rp))
    oep_results[rp] = val
    print(f"  1-in-{rp}: ${val:,.0f}")

--- AEP ---
1-in-10: $32,418,441,289
1-in-25: $49,903,720,054
1-in-50: $66,806,130,113
1-in-100: $91,011,290,739

```

```

1-in-250: $121,497,347,715

--- OEP ---
1-in-10: $23,016,706,550
1-in-25: $38,547,833,568
1-in-50: $54,329,356,096
1-in-100: $73,208,743,896
1-in-250: $104,120,846,172

fig, ax = plt.subplots(figsize=(9, 5))

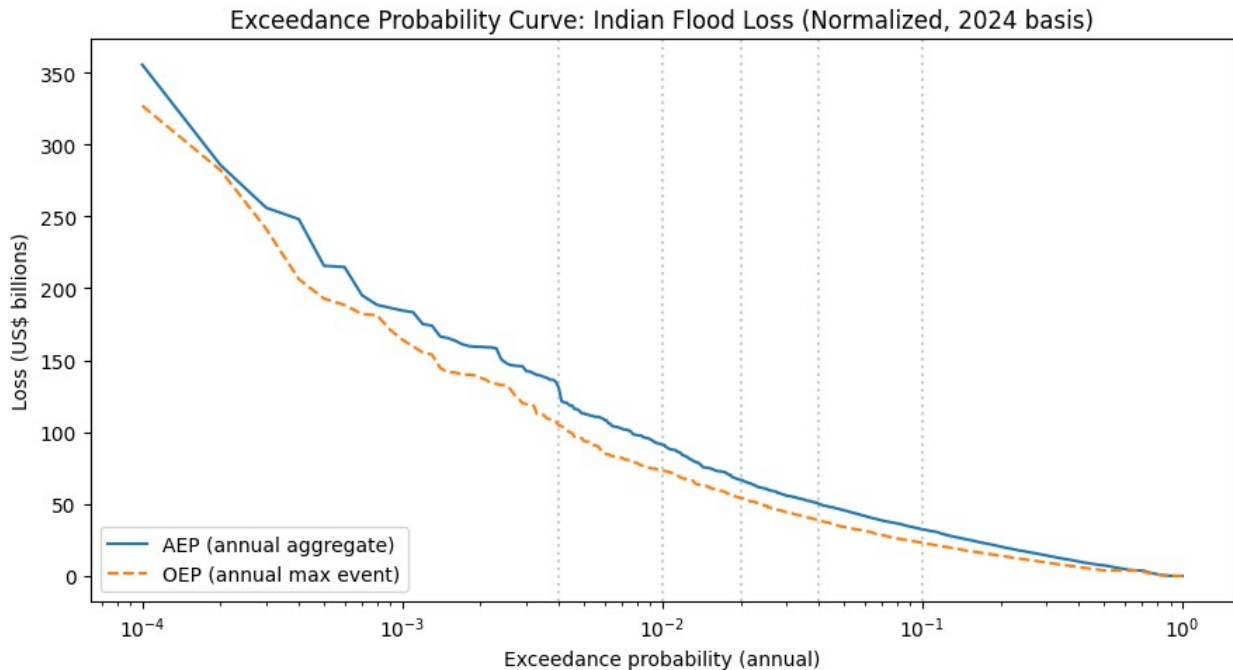
sorted_aep = np.sort(annual_totals)[::-1]
p_aep = np.arange(1, len(sorted_aep)+1) / len(sorted_aep)
ax.plot(p_aep, sorted_aep/1e9, label="AEP (annual aggregate)")

sorted_oeep = np.sort(annual_max)[::-1]
p_oeep = np.arange(1, len(sorted_oeep)+1) / len(sorted_oeep)
ax.plot(p_oeep, sorted_oeep/1e9, label="OEP (annual max event)",
        linestyle="--")

for rp in return_periods:
    ax.axvline(1/rp, color="grey", linestyle=":", alpha=0.4)

ax.set_xscale("log")
ax.set_xlabel("Exceedance probability (annual)")
ax.set_ylabel("Loss (US$ billions)")
ax.set_title("Exceedance Probability Curve: Indian Flood Loss\n(Normalized, 2024 basis)")
ax.legend()
plt.tight_layout()
plt.savefig("fig2_ep_curve.png", dpi=150)
plt.show()

```



```

N_BOOT = 1000
boot_aep = {rp: [] for rp in return_periods}
boot_oep = {rp: [] for rp in return_periods}

for b in range(N_BOOT):
    resample = np.random.choice(losses, size=len(losses),
    replace=True)
    ln_b = stats.lognorm.fit(resample, floc=0)
    thresh_b = np.percentile(resample, 75)
    exceed_b = resample[resample > thresh_b] - thresh_b
    gpd_b = stats.genpareto.fit(exceed_b, floc=0) if len(exceed_b) >=
5 else gpd_params
    exceed_prob_b = len(exceed_b) / len(resample)

    totals_b, max_b = [], []
    for _ in range(500):
        n_ev = stats.nbinom.rvs(r, p) if use_negbin else
stats.poisson.rvs(mean_events)
        if n_ev == 0:
            totals_b.append(0.0); max_b.append(0.0); continue
        tail_mask = np.random.rand(n_ev) < exceed_prob_b
        draws = np.zeros(n_ev)
        n_tail = tail_mask.sum(); n_bulk = n_ev - n_tail
        if n_bulk > 0:
            draws[~tail_mask] = np.clip(stats.lognorm.rvs(*ln_b,
size=n_bulk), 0, thresh_b)
        if n_tail > 0:
            draws[tail_mask] = thresh_b + stats.genpareto.rvs(*gpd_b,
size=n_tail)

```

```

        totals_b.append(draws.sum()); max_b.append(draws.max())

totals_b, max_b = np.array(totals_b), np.array(max_b)
for rp in return_periods:
    boot_aep[rp].append(np.percentile(totals_b, 100*(1-1/rp)))
    boot_oep[rp].append(np.percentile(max_b, 100*(1-1/rp)))

print("AEP with 90% bootstrap CI:")
for rp in return_periods:
    lo, hi = np.percentile(boot_aep[rp], [5, 95])
    print(f"  1-in-{rp}: ${aep_results[rp]:,.0f} (90% CI: ${lo:,.0f}
- ${hi:,.0f})")

print("\nOEP with 90% bootstrap CI:")
for rp in return_periods:
    lo, hi = np.percentile(boot_oep[rp], [5, 95])
    print(f"  1-in-{rp}: ${oep_results[rp]:,.0f} (90% CI: ${lo:,.0f}
- ${hi:,.0f})")

AEP with 90% bootstrap CI:
  1-in-10: $32,418,441,289 (90% CI: $19,308,940,888 -
$48,845,322,720)
  1-in-25: $49,903,720,054 (90% CI: $30,336,346,360 -
$77,136,223,551)
  1-in-50: $66,806,130,113 (90% CI: $38,315,836,380 -
$121,035,858,269)
  1-in-100: $91,011,290,739 (90% CI: $44,384,669,803 -
$210,655,714,335)
  1-in-250: $121,497,347,715 (90% CI: $51,747,707,798 -
$443,298,040,856)

OEP with 90% bootstrap CI:
  1-in-10: $23,016,706,550 (90% CI: $13,504,317,442 -
$36,163,839,834)
  1-in-25: $38,547,833,568 (90% CI: $21,092,829,330 -
$59,734,538,127)
  1-in-50: $54,329,356,096 (90% CI: $25,214,083,256 -
$107,311,248,543)
  1-in-100: $73,208,743,896 (90% CI: $26,543,103,352 -
$199,778,094,687)
  1-in-250: $104,120,846,172 (90% CI: $26,782,513,496 -
$431,980,727,847)

results_summary = pd.DataFrame({
    "return_period": return_periods,
    "AEP_loss_usd": [aep_results[rp] for rp in return_periods],
    "AEP_ci_low": [np.percentile(boot_aep[rp], 5) for rp in
return_periods],
    "AEP_ci_high": [np.percentile(boot_aep[rp], 95) for rp in
return_periods],

```

```

    "OEP_loss_usd": [oep_results[rp] for rp in return_periods],
    "OEP_ci_low": [np.percentile(boot_oep[rp], 5) for rp in
return_periods],
    "OEP_ci_high": [np.percentile(boot_oep[rp], 95) for rp in
return_periods],
})
results_summary.to_csv("ep_curve_results.csv", index=False)
d.to_csv("normalized_events.csv", index=False)

colab_files.download("ep_curve_results.csv")
colab_files.download("normalized_events.csv")
colab_files.download("fig1_normalization.png")
colab_files.download("fig2_ep_curve.png")
colab_files.download("fig3_threshold_stability.png")

results_summary
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>

{"summary": "{\n  \"name\": \"results_summary\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"return_period\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 97,\n        \"min\": 10,\n        \"max\": 250,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          25,\n          250,\n          50\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"AEP_loss_usd\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 34974284696.961945,\n        \"min\": 32418441289.42227,\n        \"max\": 121497347714.95882,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          49903720054.422195,\n          121497347714.95882,\n          66806130112.710724\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"AEP_ci_low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 121497347714.95882,\n        \"min\": 32418441289.42227,\n        \"max\": 121497347714.95882,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          49903720054.422195,\n          121497347714.95882,\n          66806130112.710724\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }\n}"}

```

```

\ "number\ ",\n          \ "std\ ": 12559589594.535463,\n          \ "min\ ":
19308940887.733173,\n          \ "max\ ": 51747707798.40066,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
30336346360.16061,\n          51747707798.40066,\n
38315836379.71988\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "AEP_ci_high\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 159314718715.30112,\n          \ "min\ ":
48845322720.36995,\n          \ "max\ ": 443298040855.6177,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
77136223550.60194,\n          443298040855.6177,\n
121035858269.29527\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "OEP_loss_usd\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 31512106387.742134,\n          \ "min\ ":
23016706550.081177,\n          \ "max\ ": 104120846172.47842,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
38547833567.62896,\n          104120846172.47842,\n
54329356095.9674\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "OEP_ci_low\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 5587355673.091283,\n          \ "min\ ":
13504317441.953497,\n          \ "max\ ": 26782513495.60852,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
21092829329.577927,\n          26782513495.60852,\n
25214083256.190865\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "OEP_ci_high\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 160838363420.16583,\n          \ "min\ ":
36163839833.66952,\n          \ "max\ ": 431980727846.7558,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
59734538127.47511,\n          431980727846.7558,\n
107311248543.06529\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          }\n          ]\n
n\ },\n          \ "type\ ": "dataframe",\n          \ "variable_name\ ": "results_summary"}

```